

Feature Ingeniering

La ingeniería de características (o feature engineering) es el proceso de usar el conocimiento del dominio para crear, transformar, seleccionar o extraer variables (características) a partir de los datos crudos, con el objetivo de que un modelo de machine learning pueda aprender de forma más efectiva.

En otras palabras: es el arte y la ciencia de convertir datos crudos en representaciones útiles para un algoritmo.

Abarca diversas actividades:

Actividad	Descripción	Ejemplo
Creación	Generar nuevas variables a partir de las existentes	edad ² , ingresos/deuda, hora del día a partir de un timestamp
Transformación	Cambiar la escala o distribución de una variable	Normalizar, estandarizar, aplicar logaritmo
Codificación	Convertir variables categóricas o textuales en numéricas	One-hot encoding, label encoding, embeddings
Extracción	Derivar características de datos complejos	Sacar bordes de una imagen, TF-IDF de un texto
Selección	Elegir las variables más relevantes y descartar el ruido	Eliminar variables correlacionadas, usar importancia de features
Reducción	Comprimir la información en menos dimensiones	PCA, t-SNE, autoencoders

El problema de clasificación del XOR

Las salidas de la compuerta XOR no son linealmente separables, por tanto no se pueden clasificar con el perceptron simple si usamos la funcion de activacion escalon (step function), la solucion es usar una red neuronal mas avanzada con otra función de activación y posiblemente mas neuronas.

Otra solución es la que exploraremos en este documento y consiste en aplicar ingeniería de características (feature ingeniering), lo que haremos será mandar nuestra funcion a una dimension mas alta aplicando una funcion que llamamos kernel.

Primero veamos la grafica de las salidas de XOR para que nos de luz sobre el problema:

Salidas del XOR

```

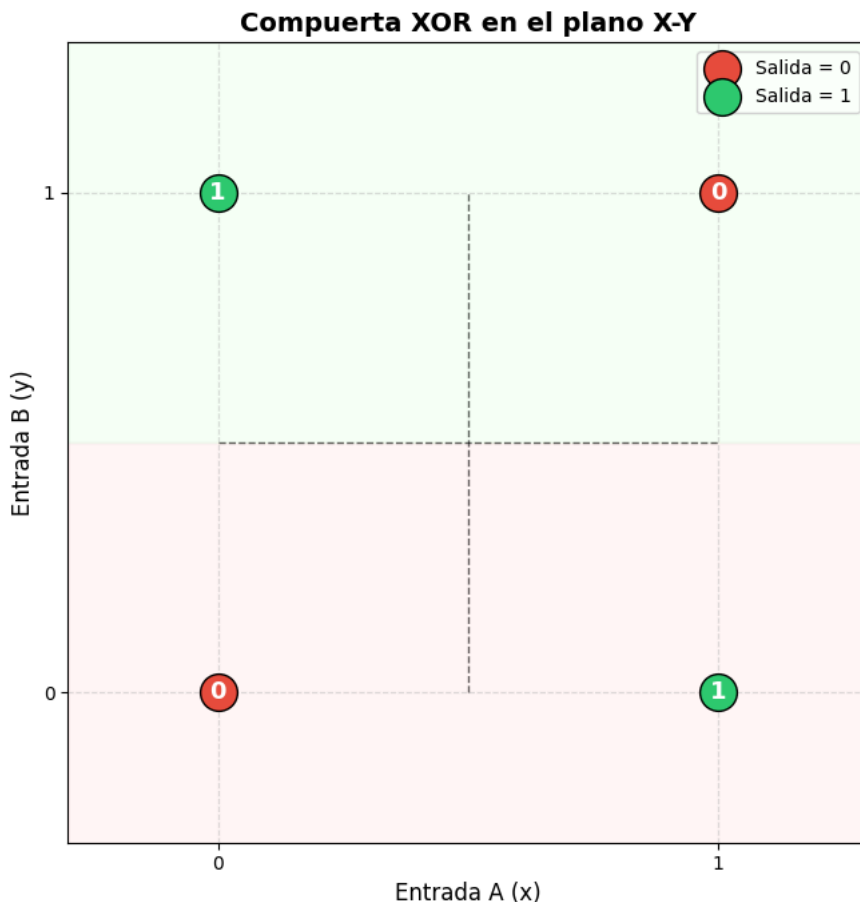
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Puntos de entrada (x, y) y su salida XOR
5 puntos = [(0, 0), (0, 1), (1, 0), (1, 1)]
6 salidas = [x ^ y for x, y in puntos] # XOR
7
8 # Separar puntos por salida
9 x0 = [p[0] for p, s in zip(puntos, salidas) if s == 0]
10 y0 = [p[1] for p, s in zip(puntos, salidas) if s == 0]
11 x1 = [p[0] for p, s in zip(puntos, salidas) if s == 1]
12 y1 = [p[1] for p, s in zip(puntos, salidas) if s == 1]
13
14 # --- Gráfica en el plano X-Y ---
15 fig, ax = plt.subplots(figsize=(7, 7))
16
17 # Regiones de fondo (opcional, para visualizar mejor)
18 ax.axhspan(-0.5, 0.5, xmin=0, xmax=1, color='#ffe6e6', alpha=0.4) # salida 0
19 ax.axhspan(0.5, 1.5, xmin=0, xmax=1, color='#e6ffe6', alpha=0.4) # salida 1
20
21 # Dibujar los puntos
22 ax.scatter(x0, y0, s=400, color='#e74c3c', edgecolor='black',
23           label='Salida = 0', zorder=3)
24 ax.scatter(x1, y1, s=400, color='#2ecc71', edgecolor='black',
25           label='Salida = 1', zorder=3)
26
27 # Etiquetar cada punto con su valor de salida
28 for (x, y), s in zip(puntos, salidas):
29     ax.annotate(str(s), (x, y), color='white', fontsize=13,
30               fontweight='bold', ha='center', va='center', zorder=4)
31
32 # Líneas separadoras (fronteras entre regiones 0 y 1)
33 ax.plot([0, 1], [0.5, 0.5], 'k--', linewidth=1, alpha=0.6)
34 ax.plot([0.5, 0.5], [0, 1], 'k--', linewidth=1, alpha=0.6)
35
36 # Configuración de los ejes
37 ax.set_xlim(-0.3, 1.3)
38 ax.set_ylim(-0.3, 1.3)

```

```

39 ax.set_xticks([0, 1])
40 ax.set_yticks([0, 1])
41 ax.set_xlabel("Entrada A (x)", fontsize=12)
42 ax.set_ylabel("Entrada B (y)", fontsize=12)
43 ax.set_title("Compuerta XOR en el plano X-Y", fontsize=14, fontweight='bold')
44 ax.grid(True, linestyle='--', alpha=0.4)
45 ax.legend(loc='upper right', fontsize=10)
46 ax.set_aspect('equal')
47
48 plt.tight_layout()
49 plt.show()

```



Como se puede apreciar en la gráfica 2D no se puede trazar una línea que separe los puntos 'verdes' de los puntos 'rojos'. **Esta función no es linealmnte separable**

✓ Solución

Elevar las salidas del XOR a una dimension superior, 3D en este caso, agregando una característica que haga los puntos linealmente separables, para este caso supongamos que las entradas del XOR (nuestras características) son (x_1, x_2) agreguemos la característica z , con lo tendremos las características en 3D (x_1, x_2, z) donde $z = x_1 * x_2$, grafiquemos estas características:

```

1 # import numpy as np
2 # import matplotlib.pyplot as plt
3 from mpl_toolkits.mplot3d import Axes3D
4
5 # Datos
6 X1 = np.array([0, 0, 1, 1])
7 X2 = np.array([0, 1, 0, 1])
8 #Z = X1 * X2 # Característica creada
9 Z=(X1 - X2) ** 2
10 Y = np.array([0, 1, 1, 0]) # Salida esperada
11
12 # Crear figura 3D
13 fig = plt.figure(figsize=(10, 8))
14 ax = fig.add_subplot(111, projection='3d')
15

```

```

16 # Colores segun la salida
17 colors = ['red' if y == 0 else 'green' for y in Y]
18
19 # Dibujar puntos
20 scatter = ax.scatter(X1, X2, Z, c=colors, s=200,
21                     edgecolors='black', linewidth=2, depthshade=True)
22
23 # Añadir etiquetas a los puntos
24 for i in range(len(X1)):
25     ax.text(X1[i], X2[i], Z[i],
26            f'({X1[i]},{X2[i]},{Z[i]})\nY={Y[i]}',
27            fontsize=10, ha='center')
28
29 # Dibujar el plano que aprende el perceptrón:
30 # Fórmula:  $-0.5 + 1 \cdot X1 + 1 \cdot X2 - 2 \cdot Z = 0 \rightarrow Z = 0.5 + 0.5 \cdot X1 + 0.5 \cdot X2$ 
31 xx, yy = np.meshgrid(np.linspace(-0.5, 1.5, 10),
32                      np.linspace(-0.5, 1.5, 10))
33 zz = 0.5 + 0.5*xx + 0.5*yy # Plano de separación
34
35 ax.plot_surface(xx, yy, zz, alpha=0.3, color='blue',
36               label='Plano de separación')
37
38 # Configurar ejes
39 ax.set_xlabel('X1', fontsize=12, fontweight='bold')
40 ax.set_ylabel('X2', fontsize=12, fontweight='bold')
41 ax.set_zlabel('Z = (X1-X2)**2', fontsize=12, fontweight='bold')
42 ax.set_title('Perceptrón XOR - Ingeniería de Características en 3D',
43             fontsize=14, fontweight='bold')
44
45 # Leyenda personalizada
46 from matplotlib.patches import Patch
47 legend_elements = [Patch(facecolor='red', label='Y = 0 (Clase 0)'),
48                   Patch(facecolor='green', label='Y = 1 (Clase 1)'),
49                   Patch(facecolor='blue', alpha=0.3, label='Plano separador')]
50 ax.legend(handles=legend_elements, loc='upper left')
51
52 # Ángulo de vista para mejor visualización
53 ax.view_init(elev=25, azim=-60)
54
55 plt.tight_layout()
56 plt.show()

```

Perceptr3n XOR - Ingenier3a de Caracter3sticas en 3D

■ $Y = 0$ (Clase 0)
■ $Y = 1$ (Clase 1)
■ Plano separador

Linealmente separables

Como puede ver ahora son linealmente separables.

¿Como fue esto?

Respuesta: al agregar la nueva caracter3stica $(x_1, x_2, x_1 \cdot x_2)$ estamos elevando nuestra funcion a 3D, agregando una nueva caracter3stica, sea $z = x_1 \cdot x_2$.

Observe que para la tabla del XOR con la nueva caracter3stica (x_1, x_2, z) tenemos:

x_1	x_2	$z = x_1 \cdot x_2$	XOR
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Es la tabla de XOR que ya conoce, solo se agrega la columna z , los valores de las coordenadas x_1, x_2 siguen siendo los mismos, pero ahora en la tercera coordenada z , al multiplicar $x_1 \cdot x_2$, la coordenada (z) hace una diferencia entre las coordenadas x_1, x_2 y en la tercera dimension (z), las salidas del XOR correspondientes a 1 estaran por encima de las salidas correspondientes a 0 que quedan 'abajo', por lo que ahora podemos encontrar un plano que separe ambas clases de salidas del XOR clases $\{0,1\}$.

Ahora son linealmente separables.

